

18YZALG - Tutorial 8: Mini-Project 1

City Metro Stops: Shortest Route with BFS

Slava, Efe, Arda

The Story: We are modeling a small city metro network where stations are vertices and train connections are edges.

Objective: Given a start station and a target station, find the shortest path based on the number of moves.

Example Case: From station A to F, our algorithm must output a path like $A \rightarrow C \rightarrow D \rightarrow F$ with a distance of 3.

Vertices (V): Each metro station is a vertex.

Edges (E): Direct train connections between stations are edges.

Weights: The graph is **unweighted**, as every move between adjacent stations counts as 1.

Directedness: The graph is **undirected**, assuming trains run in both directions between stations.

Primary Method: Breadth-First Search (BFS).

Data Structures: * **Queue:** To manage the "frontier" of the search (FIFO).

Parent Dictionary/Array: To keep track of where we came from for path reconstruction.

Why BFS?: BFS is the optimal choice for unweighted graphs because it explores all neighbors at distance k before moving to distance $k+1$, ensuring the first time we find the target, it is via the fewest possible edges.

```

def find_metro_route(graph, start,
target):
    visited = [False] * len(graph) # Tracks
visited nodes
    queue = deque([start])
    parent = {start: None} # Tracks the
path
    while queue:
        current = queue.popleft() # FIFO
Queue logic
        if current == target:
            break
        for neighbor in graph[current]:
            if neighbor is not visited:
                visited[neighbour] = True
                parent[neighbor] = current
                queue.append(neighbor)

```

```

# Reconstruct the path from parent
pointers
    if target not in parent:
        return None, None # No route
exists

```

```

path = [ ]
curr = target
while curr is not None:
    path.append(curr)
    curr = parent[curr]

```

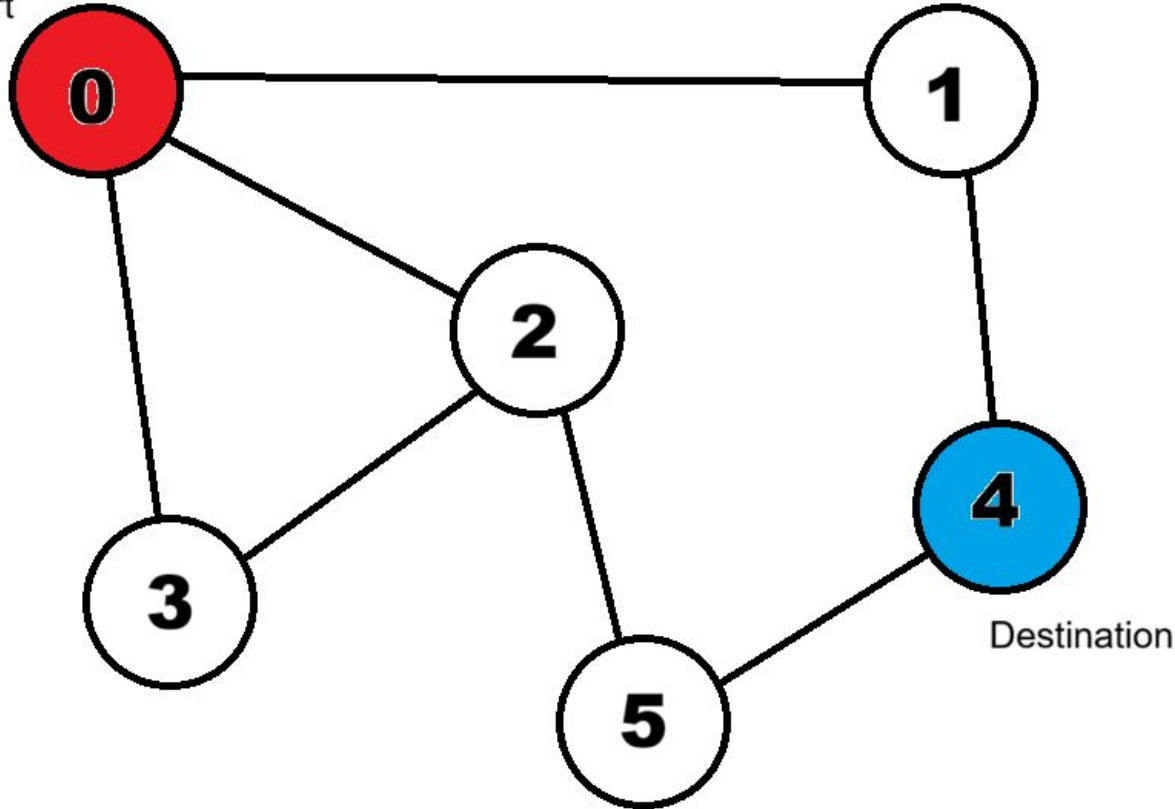
```

return reversed(path), len(path) - 1

```

BFS

Start



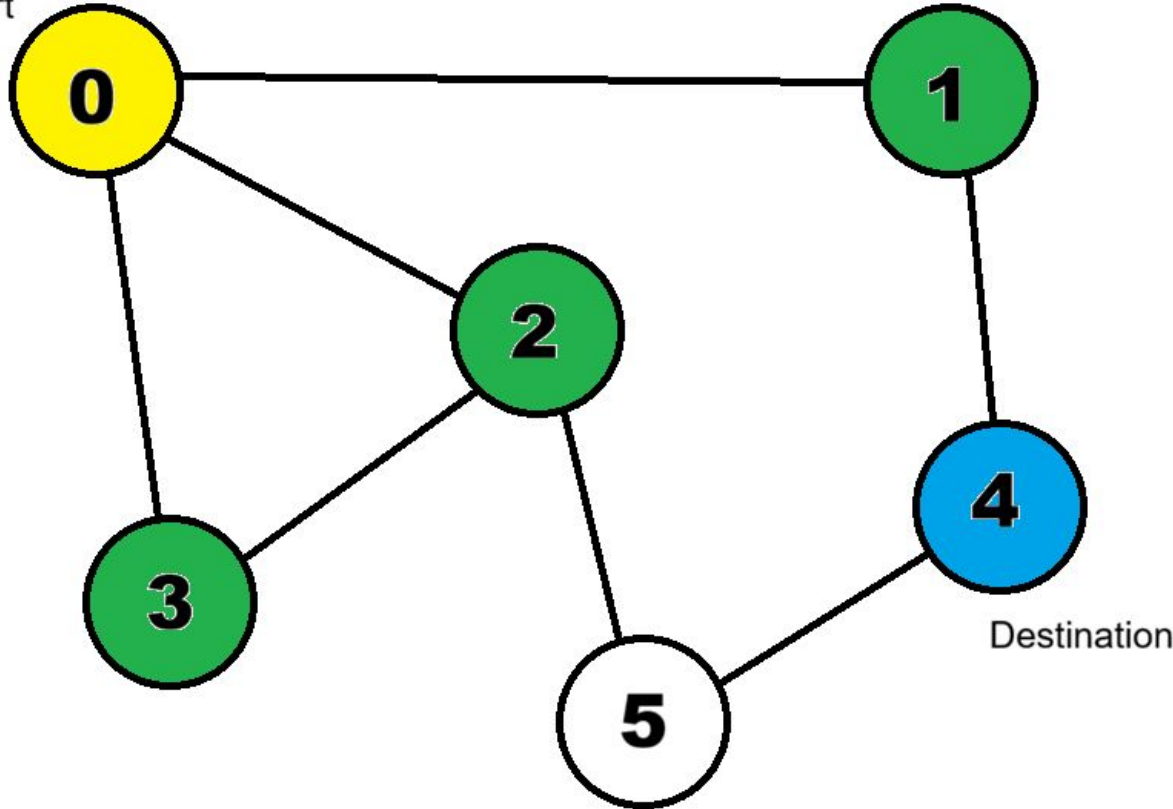
Queue:

0

parent = {
0: None
}

BFS

Start



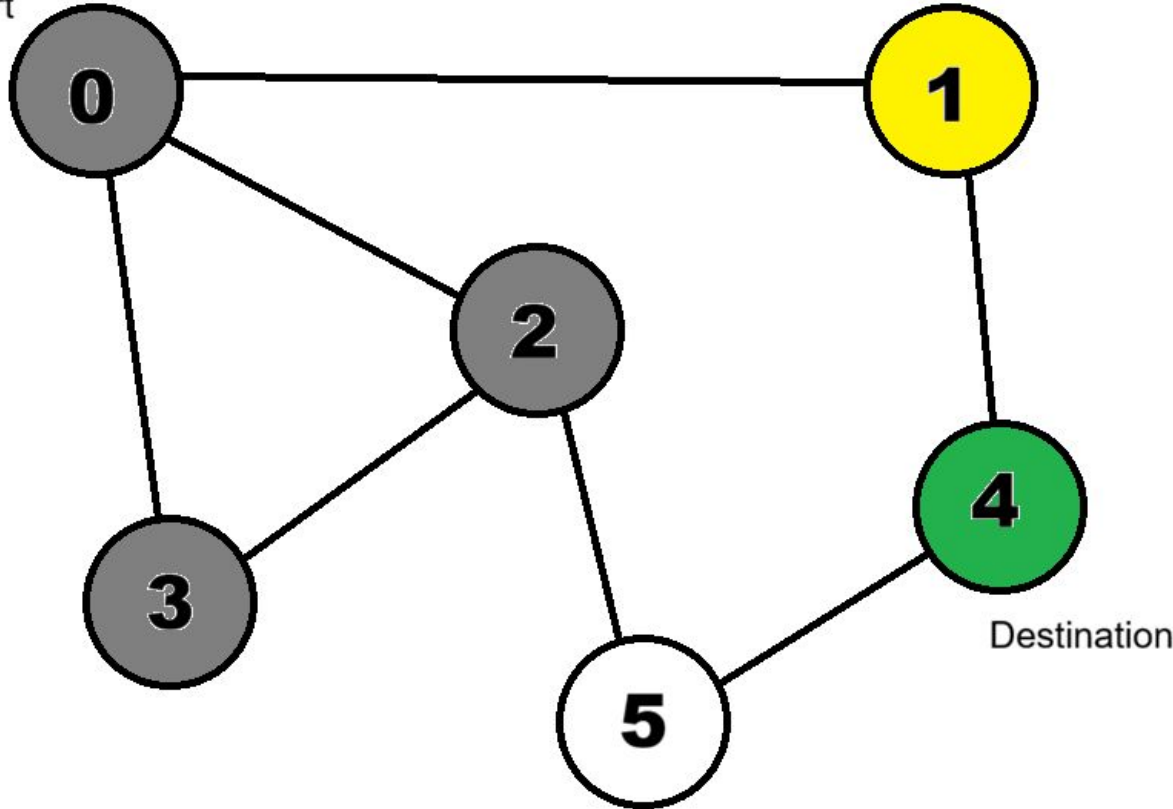
Queue:

1, 2, 3

parent = {
0: None, 1: 0,
2: 0, 3: 0
}

BFS

Start



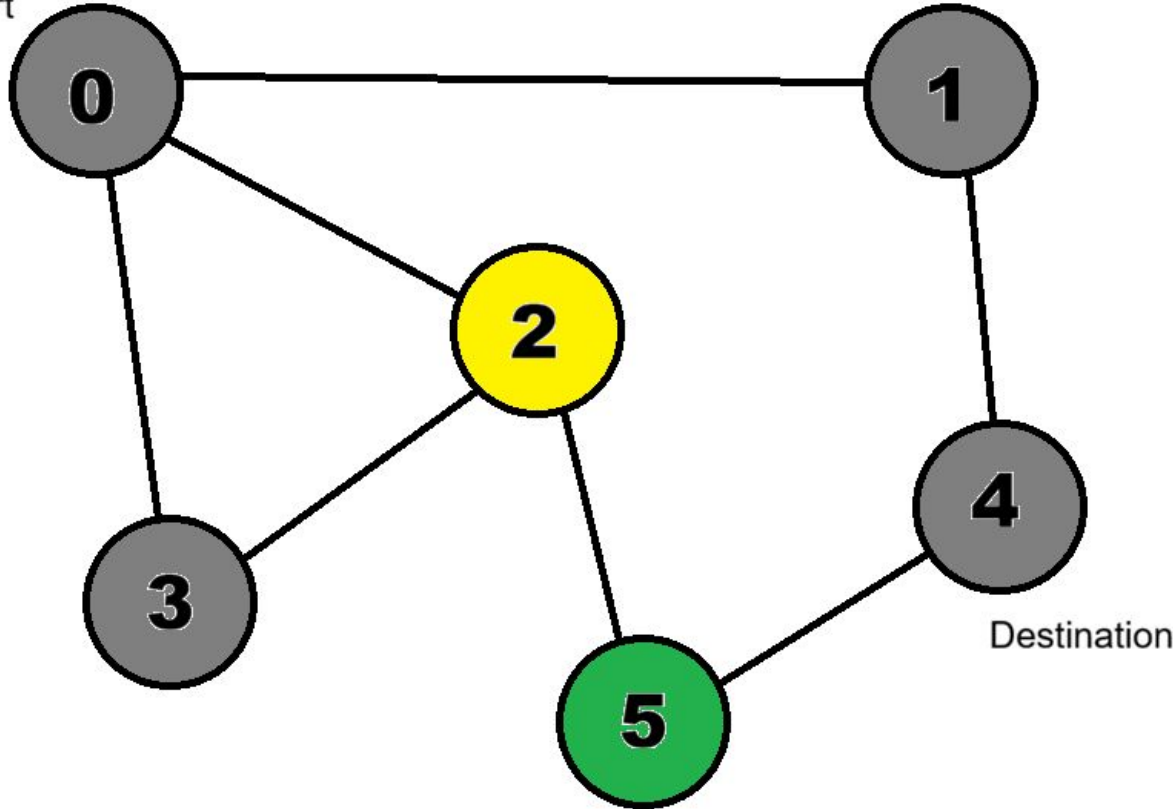
Queue:

2, 3, 4

parent = {
0: None, 1: 0,
2: 0, 3: 0, 4: 1
}

BFS

Start



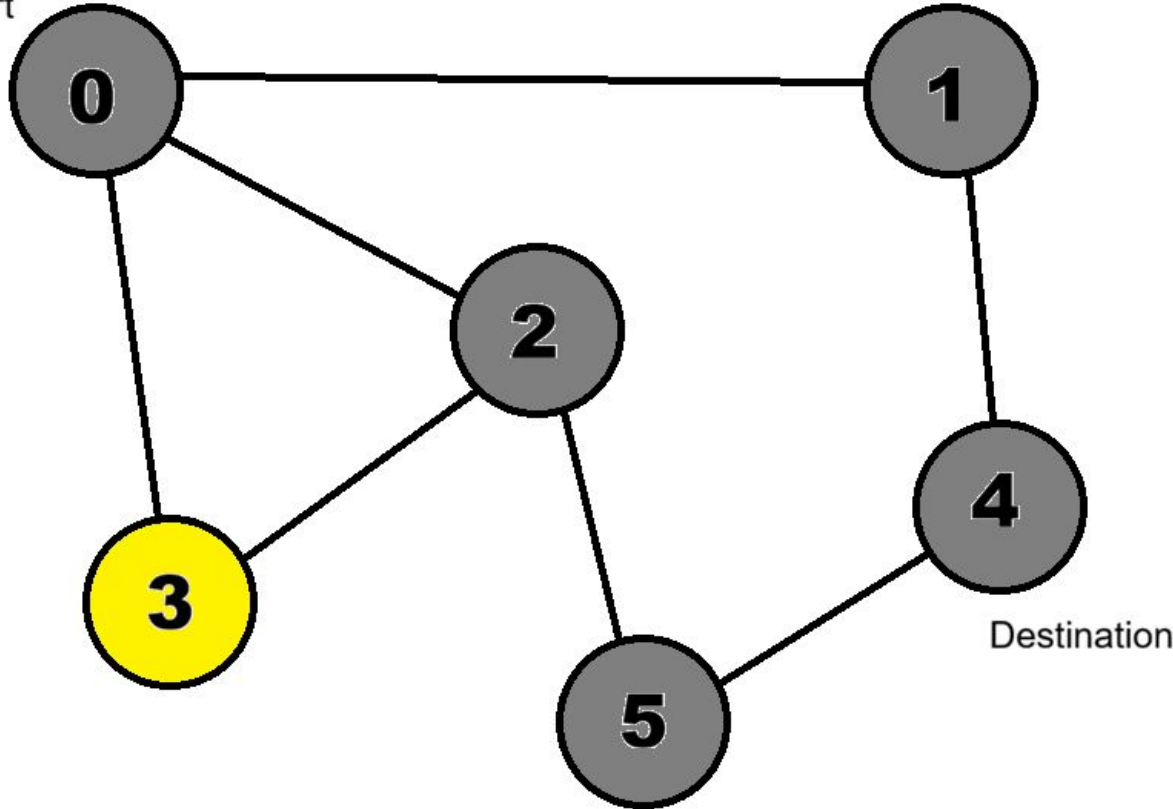
Queue:

3, 4, 5

parent = {
0: None, 1: 0,
2: 0, 3: 0, 4: 1,
5: 2
}

BFS

Start



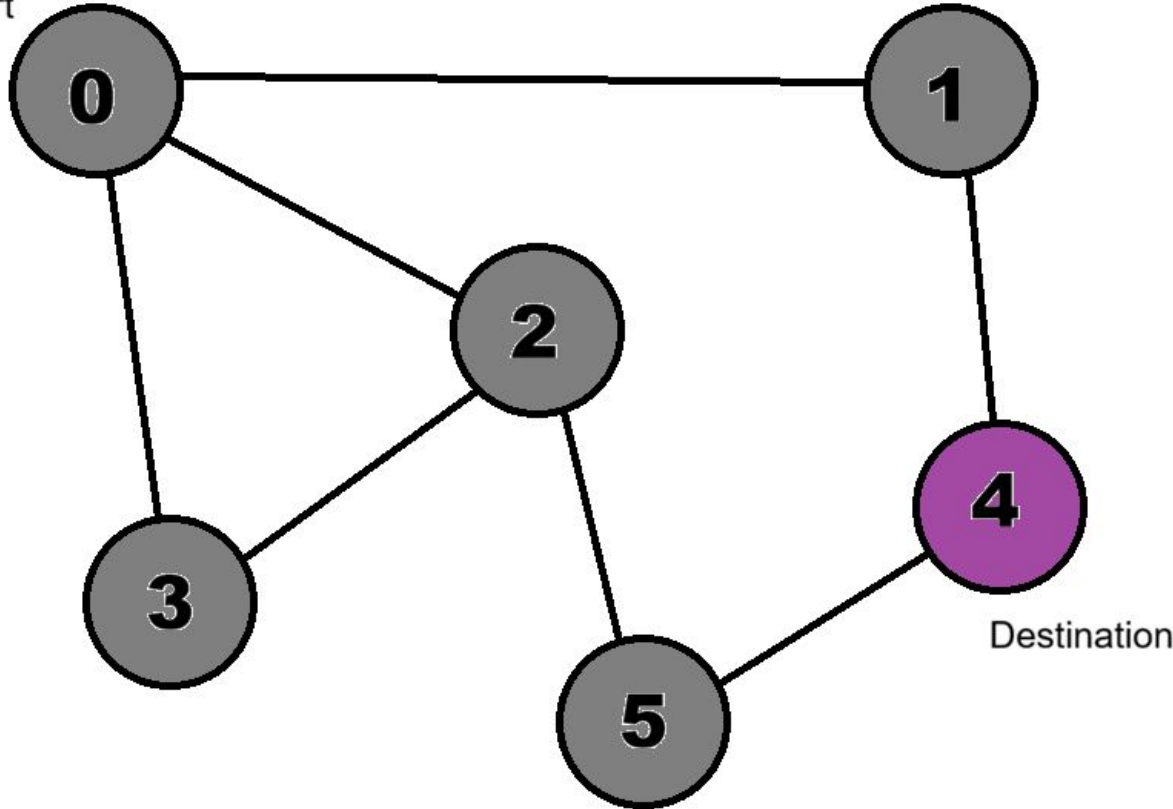
Queue:

4, 5

parent = {
0: None, 1: 0,
2: 0, 3: 0, 4: 1,
5: 2
}

BFS

Start



Queue:

4, 5

Finish!

parent = {
0: None, 1: 0,
2: 0, 3: 0, 4: 1,
5: 2
}

```

def find_metro_route(graph, start,
target):
    visited = [False] * len(graph) # Tracks
visited nodes
    stack = stack([start])
    parent = {start: None} # Tracks the
path
    while stack:
        current = stack.pop() # FIFO
Queue logic
        if current == target:
            break
    for neighbor in graph[current]:
        if neighbor is not visited:
            visited[neighbour] = True
            parent[neighbor] = current
            stack.append(neighbor)

```

```

# Reconstruct the path from parent
pointers
    if target not in parent:
        return None, None # No route
exists

```

```

path = [ ]
curr = target
while curr is not None:
    path.append(curr)
    curr = parent[curr]

```

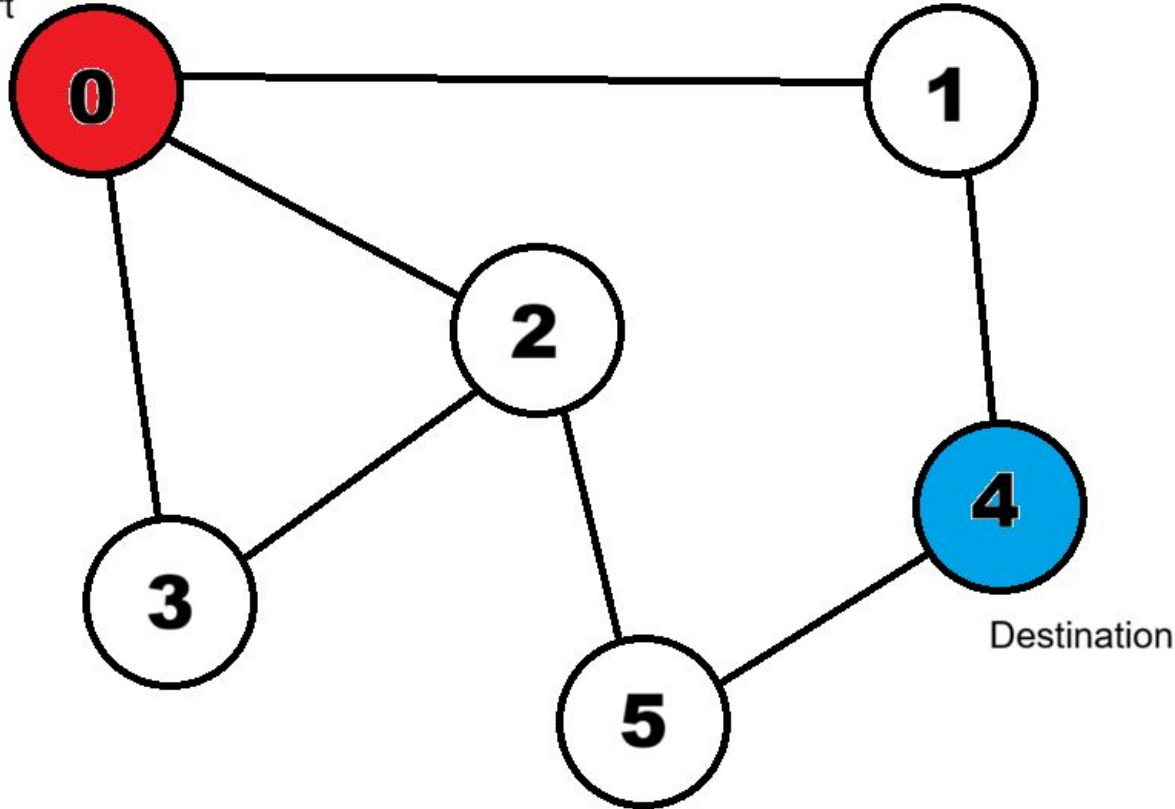
```

return reversed(path), len(path) - 1

```

DFS

Start



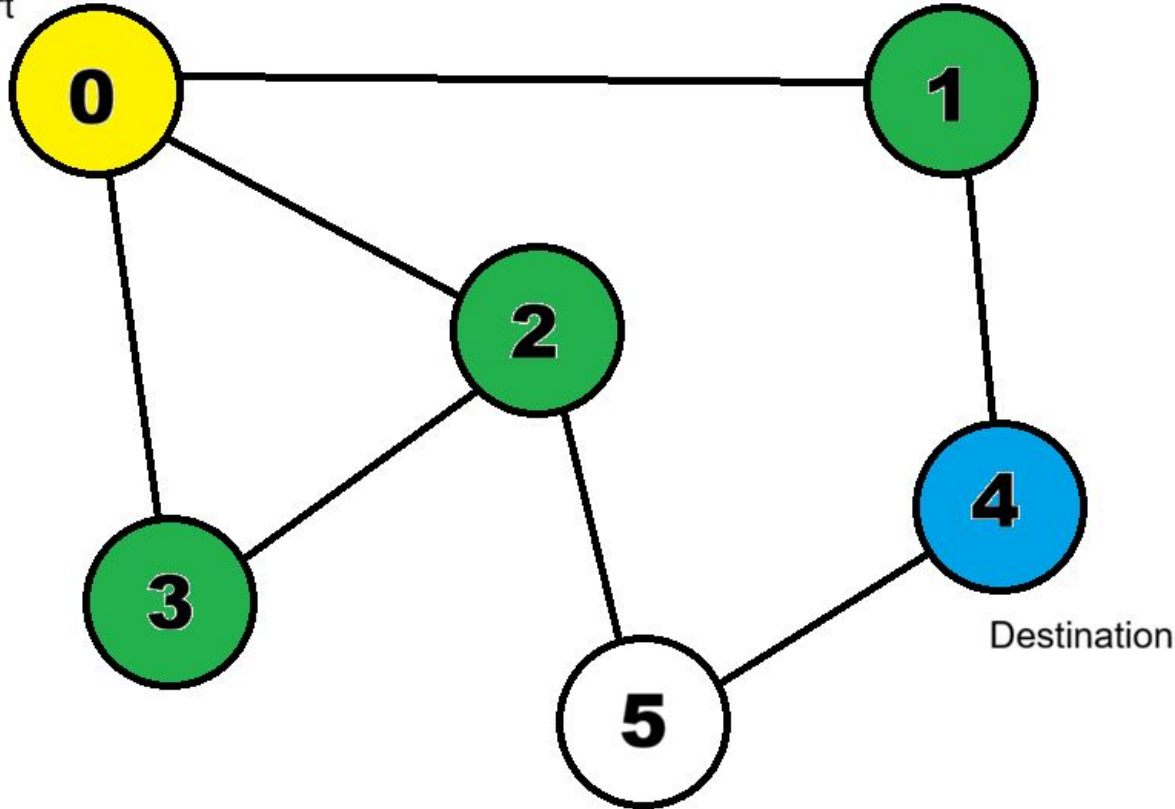
Stack:

0

parent = {
0: None
}

DFS

Start



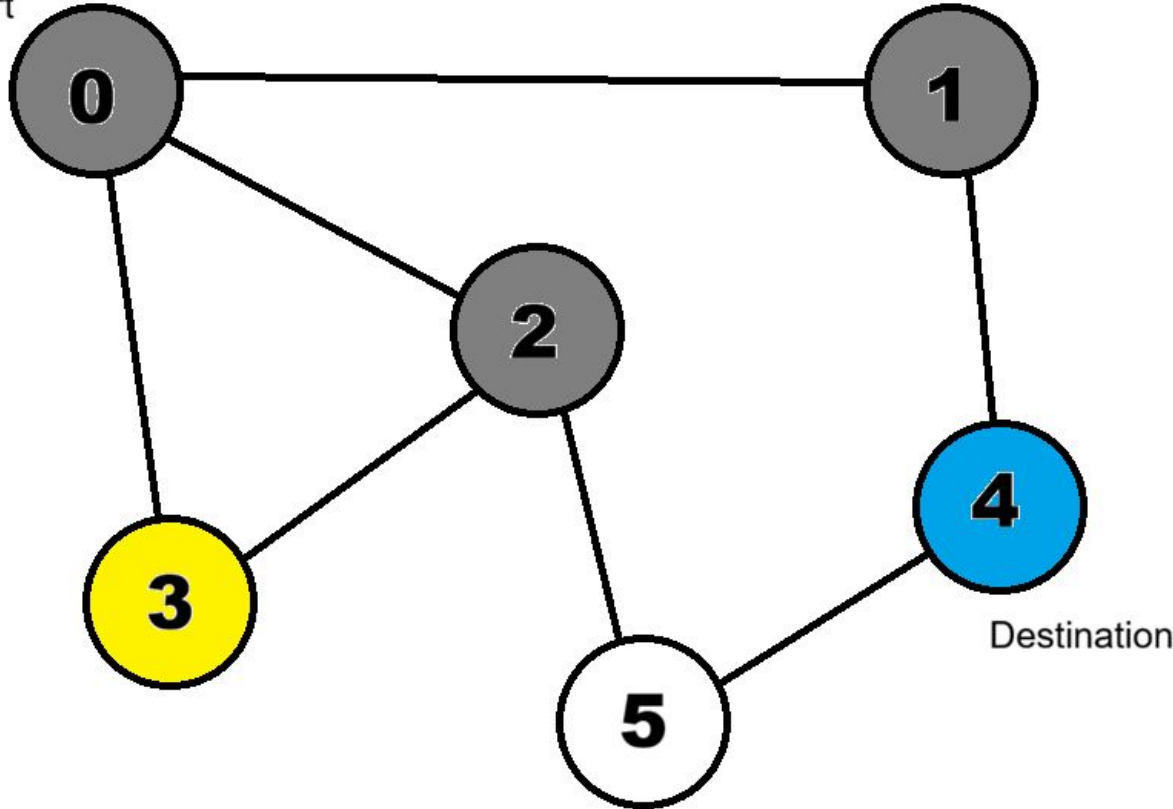
Stack:

3, 2, 1

parent = {
0: None, 1: 0,
2: 0, 3: 0
}

DFS

Start



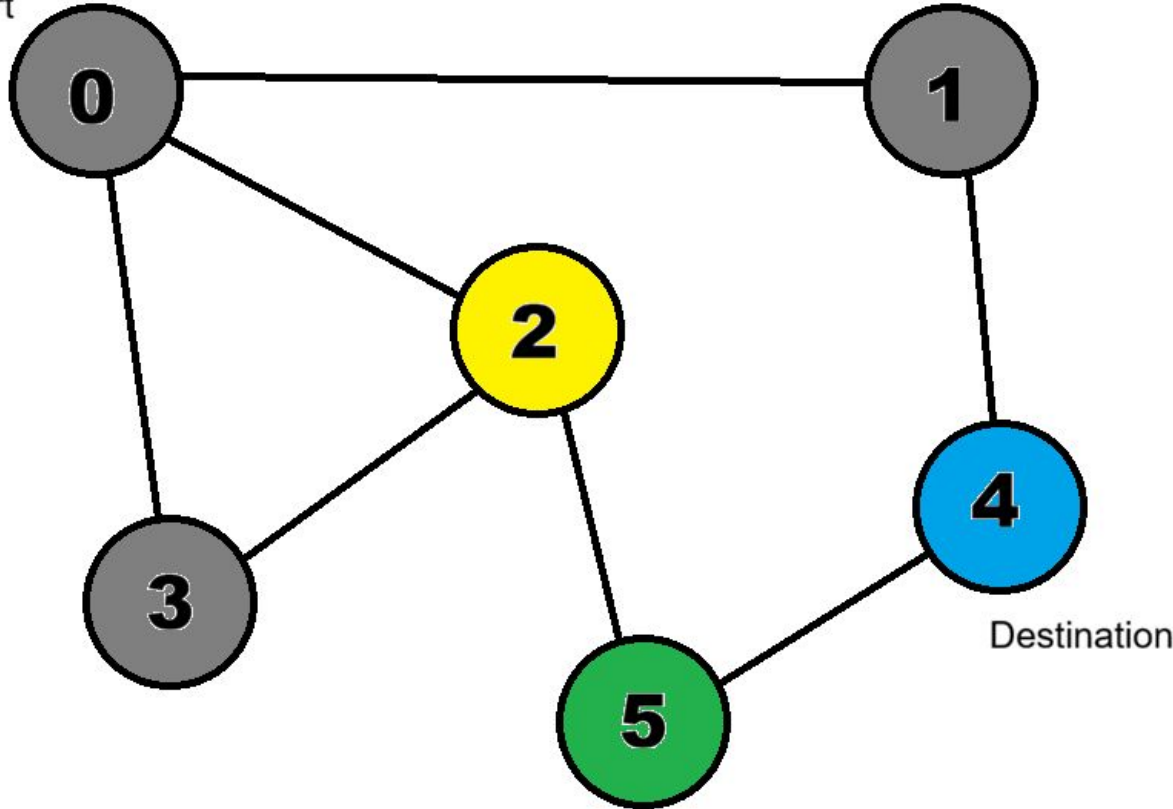
Stack:

2, 1

parent = {
0: None, 1: 0,
2: 0, 3: 0
}

DFS

Start



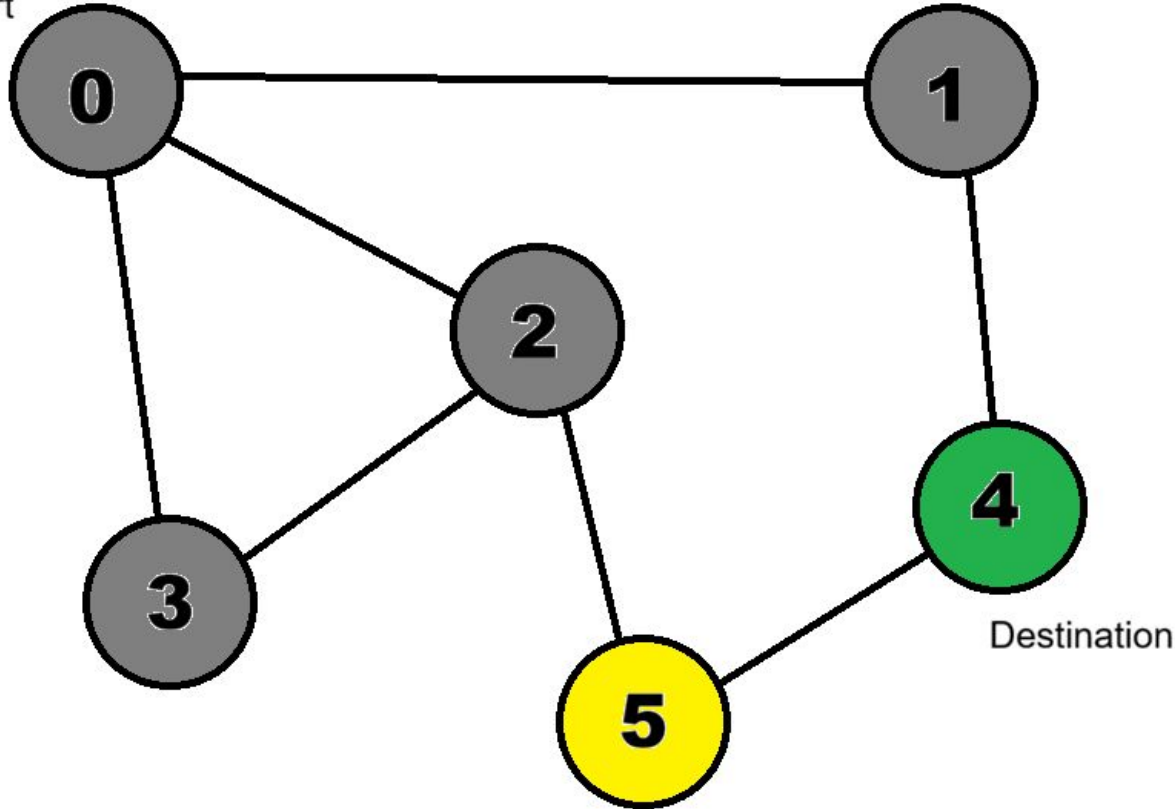
Stack:

5, 1

parent = {
0: None, 1: 0,
2: 0, 3: 0, 5: 2
}

DFS

Start



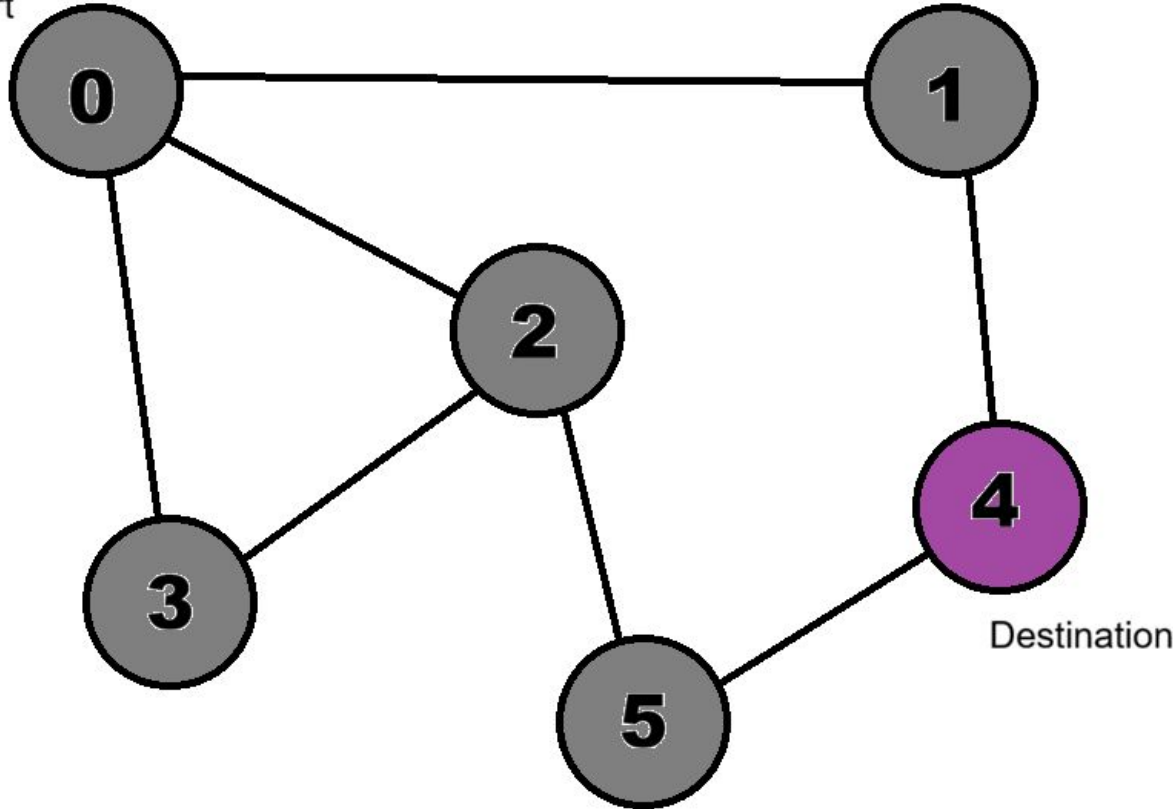
Stack:

4, 1

parent = {
0: None, 1: 0,
2: 0, 3: 0, 5: 2,
4: 5
}

DFS

Start



Stack:

4, 1

Finish!

parent = {
0: None, 1: 0,
2: 0, 3: 0, 5: 2,
4: 5
}

Concept: While searching, we store `parent[v]`, which is the station that led us to station `v`.

Rebuilding: Once the target is found, we "backtrack" from the target station to the start using these parent pointers.

Visualization: If we reach 'F' from 'D', and 'D' from 'C', and 'C' from 'A', the stored parents allow us to reverse the sequence to get $A \rightarrow C \rightarrow D \rightarrow F$.

Method B (DFS): Depth-First Search explores as far as possible along a branch before backtracking.

Key Difference: DFS may find a path to the target, but it is **not guaranteed** to be the shortest one.

Live Demo Focus: We will show an example where DFS takes a "long detour" while BFS finds the direct route

To ensure correctness, we tested the following scenarios:

1. **Normal Path:** Finding a multi-step route across the city.
2. **Start == Target:** The algorithm returns the station itself with distance 0.
3. **No Route:** If a station is isolated, the algorithm correctly reports that no path exists.

Queue Mechanics: We will demonstrate the status of the Queue for a tiny graph to show how stations enter and leave.

Performance:

- **BFS:** Visits minimum nodes required to find the shortest level.
- **DFS:** May visit significantly more nodes before finding the target.

Output: We print both the total distance and the actual list of stations.

Conclusion: BFS is the standard and most efficient algorithm for shortest-path problems in unweighted networks like basic metro systems.

Recommendation: For future expansions involving "transfer penalties" or "weighted travel times," the model should be upgraded to Dijkstra's algorithm.